

FINAL PROJECT REPORT

CSE 421

Efe Kayadelen

Student ID : 20220701088

Samet Alper Özdemir

Student ID : 20210702021

GITHUB REPO: <https://github.com/blackwh1p/CSE421-Homeworks>

Q1 – Embedded Mediterranean Fruit Fly Detection with FOMO

Unlike traditional object detection models like YOLO or MobileNet+SSD, which utilize complex bounding box regression and can be computationally expensive for microcontrollers, FOMO redefines object detection as a simplified constrained-object location problem. It functions by dividing an image into a grid and predicting the probability of an object's presence in each cell. This architectural efficiency allows for: High-speed inference, small object sensitivity, minimal memory footprint.

The chosen model utilizes a MobileNetV2 backbone with an alpha value of 0.35. This specific alpha parameter reduces the number of filters in each layer, significantly shrinking the model size and computational complexity to meet the strict memory constraints of TFLite Micro on an STM32 microcontroller.

Detecting small objects like fruit flies often involves a significant class imbalance (many more background cells other than object cells). To solve this, a “Weighted Binary Crossentropy” loss was implemented: Positional weighting, a “POS_WEIGHT” of 10.0 is applied to positive detections. The working principle of “POS_WEIGHT” is to proportionally scale the loss value of positive samples during training, ensuring the model's optimization process treats the minority class as more significant than the majority class. This forces the model to prioritize finding the flies (reducing false negatives) while preventing the "dirt" or background noise from being classified as targets (controlling false positives).

To ensure the model performs reliably under varying field conditions, a custom augmentation pipeline was integrated into the training loop:

- Spatial Transforms: Random horizontal and vertical flips.
- Environmental Simulation: Random brightness adjustments (scaling between 0.7 and 1.3) simulate the lighting changes found in Mediterranean orchards.

Training and regularization of code:

- Inference Efficiency: The model extracts features from an intermediate layer (block_6_expand_relu) to maintain a high-resolution feature map (12x12) required for precise location tracking.
- Overfitting Control: A dropout layer (0.3) and a slower learning rate (0.0005) were utilized to ensure the model generalizes well to unseen test data.
- Optimization: The Adam optimizer was used for 100 epochs to ensure convergence of the weighted loss function.

Table 1 Training Code for FOMO application

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import cv2
import os
```

```

import json

# --- CONFIGURATION ---
DATASET_DIR = "dataset"
IMG_SIZE = 96
ORIGINAL_SIZE = 400
GRID_SIZE = 12
EPOCHS = 100          # Increased epochs for better convergence
BATCH_SIZE = 16

# --- CUSTOM WEIGHTED LOSS ---
# Balanced weight: High enough to find flies, low enough to ignore dirt.
def weighted_binary_crossentropy(y_true, y_pred):
    POS_WEIGHT = 10.0 # Reduced from 100 to 10 to reduce false positives
    bce = tf.keras.backend.binary_crossentropy(y_true, y_pred)
    weight_vector = y_true * POS_WEIGHT + (1.0 - y_true)
    return tf.reduce_mean(weight_vector * bce)

def augment_image(img, label):
    """
    Applies random brightness, contrast, and flips to make the model robust.
    """
    # 1. Random Flip Left-Right
    if np.random.rand() > 0.5:
        img = cv2.flip(img, 1)
        label = np.fliplr(label)

    # 2. Random Flip Up-Down
    if np.random.rand() > 0.5:
        img = cv2.flip(img, 0)
        label = np.flipud(label)

    # 3. Random Brightness (Simulates glare/shadows)
    # Convert to float for math, then clip back
    brightness = np.random.uniform(0.7, 1.3)
    img = np.clip(img * brightness, 0.0, 1.0)

    return img, label

def load_data(subset):
    images = []
    labels = []

    label_path = os.path.join(DATASET_DIR, f"{subset}_labels.json")
    with open(label_path, 'r') as f:
        file_list = json.load(f)['files']

    print(f"[INFO] Loading {subset} dataset...")

    img_scale = IMG_SIZE / ORIGINAL_SIZE

    for entry in file_list:
        img_path = os.path.join(DATASET_DIR, subset, entry['path'])
        img = cv2.imread(img_path)
        if img is None: continue

        # Resize
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

```

```

img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
img_norm = img.astype(np.float32) / 255.0

# Label Generation
label_grid = np.zeros((GRID_SIZE, GRID_SIZE, 1), dtype=np.float32)

for bbox in entry.get('boundingBoxes', []):
    cx = (bbox['x'] + bbox['width'] / 2) * img_scale
    cy = (bbox['y'] + bbox['height'] / 2) * img_scale

    gx = int(cx / 8) # 96/12 = 8
    gy = int(cy / 8)

    if 0 <= gx < GRID_SIZE and 0 <= gy < GRID_SIZE:
        label_grid[gy, gx, 0] = 1.0

# Add Original
images.append(img_norm)
labels.append(label_grid)

# Add Augmented Version (Only for Training)
if subset == "train":
    aug_img, aug_label = augment_image(img_norm, label_grid)
    images.append(aug_img)
    labels.append(aug_label)

return np.array(images), np.array(labels)

# --- EXECUTION ---
x_train, y_train = load_data("train")
x_test, y_test = load_data("test")

# Model Definition (MobileNetV2 alpha 0.1 for speed/size)
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    include_top=False,
    alpha=0.35, # Smaller alpha reduces model size further
    weights='imagenet'
)

try:
    fomo_layer = base_model.get_layer('block_6_expand_relu').output
except:
    fomo_layer = base_model.output

x = layers.Conv2D(32, (1, 1), padding='same', activation='relu')(fomo_layer)
x = layers.Dropout(0.3)(x) # Higher dropout to prevent overfitting
output = layers.Conv2D(1, (1, 1), padding='same', activation='sigmoid')(x)

model = models.Model(inputs=base_model.input, outputs=output)

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005), # Slower learning rate
    loss=weighted_binary_crossentropy,
    metrics=['accuracy']
)

print("[INFO] Training with Augmentation...")

```

```

model.fit(
    x_train, y_train,
    epochs=EPOCHS,
    validation_data=(x_test, y_test),
    batch_size=BATCH_SIZE,
    shuffle=True
)

if not os.path.exists("models"): os.makedirs("models")
model.save("models/fruitfly_fomo.h5")
print("[SUCCESS] New Robust Model Saved.")

```

To execute the FOMO model on the TFLite Micro framework, the model was converted from 32-bit floating-point weights to 8-bit integers (INT8). This optimization is critical for the STM32 for two reasons: Memory reduction and hardware acceleration. Because standard INT8 quantization can lead to accuracy loss, a representative dataset generator was implemented. This function fed 100 real images from the Mediterranean fruit fly dataset into the converter:

- **Calibration:** This allowed the TFLite converter to measure the dynamic range of the activations.
- **Precision Preservation:** By "seeing" real-world data during conversion, the model maintains high accuracy even after the weights are compressed to 8-bit integers.

Since microcontrollers cannot read .tflite files from a file system, the binary data was converted into a C++ byte array (model_data.h).

Table 2 Model Converter Code to TFLite Micro

```

import tensorflow as tf
import numpy as np
import os
import cv2

MODEL_PATH = "models/fruitfly_fomo.h5"
DATASET_DIR = "dataset/train"
OUTPUT_HEADER = "../model_data.h"
OUTPUT_TFLITE = "models/fruitfly_fomo_quant.tflite"
IMG_SIZE = 96

if not os.path.exists(MODEL_PATH):
    print(f"[ERROR] Model not found. Run training.py first.")
    exit()

# --- CRITICAL FIX: compile=False ---
# This prevents the "Unknown loss function" error
print("[INFO] Loading Keras model...")
model = tf.keras.models.load_model(MODEL_PATH, compile=False)

def representative_dataset_gen():
    files = [f for f in os.listdir(DATASET_DIR) if f.endswith((''.jpg', '.png', '.jpeg'))]
    for file_name in files[:100]:
        img = cv2.imread(os.path.join(DATASET_DIR, file_name))
        if img is None: continue
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

```

```

        img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
        img = img.astype(np.float32) / 255.0
        img = np.expand_dims(img, axis=0)
        yield [img]

print("[INFO] Converting to INT8 TFLite...")
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_dataset_gen
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8

tflite_model = converter.convert()

with open(OUTPUT_TFLITE, "wb") as f:
    f.write(tflite_model)
print(f"[SUCCESS] Saved {OUTPUT_TFLITE}")

# Generate Header for STM32
def create_header(data, var_name):
    header = f"/ * Generated by convert_to_tflite.py */\n"
    header += f"#ifndef MODEL_DATA_H\n#define MODEL_DATA_H\n"
    header += f"#include <stdint.h>\n"
    header += f"alignas(16) const unsigned char {var_name} [] = {\n "
    for i, byte in enumerate(data):
        header += f"0x{byte:02x}, "
        if (i + 1) % 12 == 0: header += "\n "
    header += "\n};\n"
    header += f"const int {var_name}_len = {len(data)};\n"
    header += f"#endif\n"
    return header

print(f"[INFO] Writing {OUTPUT_HEADER}...")
with open(OUTPUT_HEADER, "w") as f:
    f.write(create_header(tflite_model, "fruitfly_fomo_model"))

print("[SUCCESS] Header file generated.")

```

Mbed Studio side of the implementation utilizes the TensorFlow Lite for Microcontrollers (TFLite Micro) framework. The core of the system is the MicroInterpreter, which handles the execution of the quantized FOMO model.

- **Memory Management:** A “tensor_arena” of 250 KB was allocated to provide enough static RAM for the MobileNetV2-based model's activations and intermediate scratchpad buffers.
- **Operation Resolver:** A “MicroMutableOpResolver” was configured to register only the specific operators used by the MobileNetV2 alpha 0.35 architecture. This "selective registration" significantly reduces the final binary size on the microcontroller's Flash memory.

To facilitate real-time testing and data acquisition, a custom serial communication protocol was established via the ST-Link Virtual COM Port (115200 baud).

Because the FOMO model outputs a spatial heatmap (12x12 grid) rather than a single classification, the C++ code implements a specialized post-processing loop:

- **Threshold Filtering:** The code iterates through each of the 144 grid cells. If a cell's score exceeds the “SCORE_THRESHOLD” (set to 20, approximately 30% probability), it is marked as a detection.
- **Coordinate Extraction:** The grid index is converted back into (x, y) spatial coordinates.
- **Concurrent Reporting:** For every detected fruit fly, the board sends a formatted string over serial, allowing the PC to visualize multiple targets simultaneously.
- **Final Summary:** An “END:count” signal is sent at the end of each frame to synchronize the host application.

Table 3 Mbed Studio Code for FOMO Application

```

/*
 * FOMO Object Detection - Multi-Object Reporting Version
 * Target Board: STM32F746G-Discovery
 * Framework: TensorFlow Lite for Microcontrollers
 *
 * Protocol:
 * 1. Board sends '#' (Ready)
 * 2. PC sends image data (96x96 bytes)
 * 3. Board runs inference
 * 4. Board sends "DET:x,y,score\n" for every detected object
 * 5. Board sends "END:count\n" to finish
 */

#include "mbed.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/schema/schema_generated.h"
#include "model_data.h"

// --- HARDWARE SETTINGS ---
// USBTX and USBRX are the pins for the ST-Link Virtual COM Port
static UnbufferedSerial pc(USBTX, USBRX, 115200);
DigitalOut led(LED1);

// --- MEMORY SETTINGS ---
// Tensor Arena for TensorFlow allocations
// 250KB is sufficient for MobileNetV2 alpha 0.35
const int kTensorArenaSize = 250 * 1024;
alignas(16) uint8_t tensor_arena[kTensorArenaSize];

// --- CONSTANTS ---
// FOMO output is a 12x12 grid for 96x96 input
const int GRID_WIDTH = 12;
const int GRID_HEIGHT = 12;

// Threshold for detection (int8 range: -128 to 127)
// -50 is approx 0.3 probability. Adjust higher (e.g., 0) to reduce false positives.
const int8_t SCORE_THRESHOLD = 20;

// Helper function to send string over serial
void serial_write_str(const char* str) {

```

```

    pc.write(str, strlen(str));
}

int main() {
    // Startup Blink
    led = 1; thread_sleep_for(200); led = 0; thread_sleep_for(200);

    // --- TFLITE SETUP ---
    // Resolver: Registers the operators used by the model
    static tflite::MicroMutableOpResolver<20> resolver;

    // Convolution & Geometry Ops
    resolver.AddConv2D();
    resolver.AddDepthwiseConv2D();
    resolver.AddReshape();
    resolver.AddPad();
    resolver.AddConcatenation(); // Important for MobileNet blocks

    // Activation Ops
    resolver.AddRelu();
    resolver.AddRelu6(); // Critical: MobileNetV2 uses ReLU6
    resolver.AddSoftmax();
    resolver.AddLogistic(); // Sigmoid (Last layer for FOMO)

    // Math Ops
    resolver.AddAdd();
    resolver.AddMul();
    resolver.AddMean();
    resolver.AddFullyConnected();

    // Quantization Ops
    resolver.AddQuantize();
    resolver.AddDequantize();

    // Load Model
    const tflite::Model* model = tflite::GetModel(fruitfly_fomo_model);
    if (model->version() != TFLITE_SCHEMA_VERSION) {
        serial_write_str("Error: Model Schema Version Mismatch!\n");
        while(1) { led = !led; thread_sleep_for(50); }
    }

    // Instantiate Interpreter
    static tflite::MicroInterpreter interpreter(
        model, resolver, tensor_arena, kTensorArenaSize);

    // Allocate Tensors
    if (interpreter.AllocateTensors() != kTfLiteOk) {
        serial_write_str("Error: AllocateTensors() Failed! Increase Arena Size.\n");
        while(1) { led = !led; thread_sleep_for(500); }
    }

    TfLiteTensor* input = interpreter.input(0);
    TfLiteTensor* output = interpreter.output(0);

    serial_write_str("--- STM32 FOMO Multi-Detect Ready ---\n");

    // --- MAIN LOOP ---
    while (true) {

```



```

// 1. Send Handshake
char handshake = '#';
pc.write(&handshake, 1);

// 2. Receive Image
// Input expects signed int8 [-128, 127]
int total_bytes = input->bytes;
int bytes_received = 0;
int8_t* input_ptr = input->data.int8;

while (bytes_received < total_bytes) {
    if (pc.readable()) {
        pc.read(&input_ptr[bytes_received], 1);
        bytes_received++;
    }
}

led = 1; // Processing indicator ON

// 3. Run Inference
TfLiteStatus invoke_status = interpreter.Invoke();
if (invoke_status != kTfLiteOk) {
    serial_write_str("Error: Invoke Failed!\n");
    led = 0;
    continue;
}

// 4. Process Results & Send Multiple Detections
int8_t* results = output->data.int8;
int detection_count = 0;
char buffer[64];

// Iterate through the 12x12 grid
for (int y = 0; y < GRID_HEIGHT; y++) {
    for (int x = 0; x < GRID_WIDTH; x++) {
        // Calculate index in 1D array
        // If model has multiple classes, index = (y * width + x) * num_classes + class_id
        int index = y * GRID_WIDTH + x;
        int8_t score = results[index];

        // If confidence is above threshold, report it
        if (score > SCORE_THRESHOLD) {
            // Format: DET:x,y,score
            int len = sprintf(buffer, "DET:%d,%d,%d\n", x, y, score);
            pc.write(buffer, len);
            detection_count++;
        }
    }
}

// 5. Send End Signal
int end_len = sprintf(buffer, "END:%d\n", detection_count);
pc.write(buffer, end_len);

led = 0; // Processing indicator OFF
}
}

```

Table 4 FOMO Application Results

=====	
Total Ground Truth Objects	: 92
Total Predictions Made	: 138

True Positives (TP)	: 65
False Positives (FP)	: 42
False Negatives (FN)	: 27

Detection Accuracy	: 48.51%
Precision	: 60.75%
Recall	: 70.65%
F1 Score	: 0.65
=====	

After testing with the Python code on the independent test data, the results confirm that both implemented models successfully met the project objectives for embedded performance. The FOMO model achieved an F1-Score of 0.65 with a high Recall of 70.65%. These results confirm the model's high sensitivity for locating Mediterranean fruit flies within a constrained 12x12 grid. With 65 True Positives identified out of 92 ground truth objects, the system effectively balances detection capability with the strict memory limits of the TFLite Micro environment.

Q2 – Embedded Plant Disease Classification: MobileNet Deployment on TFLite Micro

Plant diseases significantly threaten global food security, causing up to 40% in annual crop losses. This part of the project implements an Edge AI solution for real-time disease recognition on low-power hardware using TFLite Micro. By processing data locally on a microcontroller, the system provides immediate diagnostics in remote areas without needing internet connectivity.

MobileNet was selected for this task because it is specifically engineered for high-efficiency "TinyML" applications.

To ensure the model generalizes well to real-world plant images, a data pipeline was implemented:

- Automatic Dataset Splitting: The raw training data was split into 80% training and 20% validation subsets to monitor for overfitting in real-time.
- On-the-Fly Augmentation: To make the model invariant to lighting and orientation, a preprocessing layer applies random flips, rotations (10%), and zooms (10%) to every image during training.

- **Performance Optimization:** The “tf.data.AUTOTUNE” and “cache()” methods were used to prefetch images into memory, ensuring the CPU never bottlenecks the training process.

MobileNetV2 backbone settings:

- **Weight Initialization:** The model uses weights pre-trained on the ImageNet dataset, which already contains features for edges, textures, and shapes.
- **Alpha Parameter (0.35):** A critical optimization for TFLite Micro was setting alpha=0.35. This "width multiplier" reduces the number of filters in each layer, creating a "Tiny" version of MobileNet that fits perfectly within the STM32's memory.
- **Input Normalization:** A rescaling layer maps the pixel values from [0, 255] to the [-1, 1] range required by the MobileNetV2 architecture.

To achieve maximum accuracy, a two-stage training approach was employed:

- **Stage 1 - Feature Extraction:** The base MobileNet layers were frozen. Only the final classification head was trained to adapt to the specific plant disease classes.
- **Stage 2 - Fine-Tuning:** The model was partially "unfrozen" from layer 100 onwards. By re-training with a very low learning rate, the deeper layers were allowed to adjust their high-level feature detectors to the subtle visual symptoms of plant diseases without destroying the pre-trained weights.

Architecture Summary for training:

- **Input Layer:** 96x96x3 (RGB).
- **Global Average Pooling:** Converts the 2D feature maps into a 1D vector, significantly reducing the parameter count compared to "Flatten" layers.
- **Dropout (0.2):** Introduced before the final layer to prevent the model from over-relying on specific pixels, enhancing robustness.
- **Output Layer:** A Dense layer with a Softmax activation, outputting the probability distribution across the detected plant disease classes.

Table 5 Training Code for Plant Disease Recognition

```
import tensorflow as tf
from tensorflow.keras import layers, models, Input
import os
import numpy as np

# --- CONFIGURATION ---
DATASET_DIR = "dataset"
TRAIN_DIR = os.path.join(DATASET_DIR, 'train')
IMG_SIZE = 96          # MobileNetV2 standard minimum is 96x96
BATCH_SIZE = 32
EPOCHS = 10           # Transfer learning converges faster
MODEL_PATH = "models/plant_diseases_mobilenet.h5"
VALIDATION_SPLIT = 0.2
```

```

# --- 1. DATA LOADING (AUTO SPLIT) ---
print(f"[INFO] Loading Training Data ({(1-VALIDATION_SPLIT)*100}%) from {TRAIN_DIR}...")

train_ds = tf.keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
    validation_split=VALIDATION_SPLIT,
    subset="training",
    seed=123,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    label_mode='int'
)

print(f"[INFO] Loading Validation Data ({VALIDATION_SPLIT*100}%) from {TRAIN_DIR}...")

val_ds = tf.keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
    validation_split=VALIDATION_SPLIT,
    subset="validation",
    seed=123,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    label_mode='int'
)

class_names = train_ds.class_names
NUM_CLASSES = len(class_names)
print(f"[INFO] Detected {NUM_CLASSES} classes: {class_names}")

# Optimization
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)

# Data Augmentation
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])

# --- 2. MOBILENET V2 ARCHITECTURE ---
# We use alpha=0.35 for the smallest possible model size
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    include_top=False,
    weights='imagenet',
    alpha=0.35
)

# Freeze the base model to keep pre-trained weights
base_model.trainable = False

# Build the final model
inputs = Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = data_augmentation(inputs)

# MobileNetV2 expects inputs in range [-1, 1]

```

```

x = tf.keras.layers.Rescaling(1./127.5, offset=-1)(x)

# Pass through MobileNet
x = base_model(x, training=False)

# Classification Head
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.2)(x)
outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

model = models.Model(inputs, outputs, name="MobileNetV2_Tiny")
model.summary()

# --- 3. TRAIN ---
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)

# --- 4. FINE TUNING (OPTIONAL BUT RECOMMENDED) ---
# Unfreeze the last few layers for better accuracy
print("[INFO] Fine-tuning model...")
base_model.trainable = True
# Fine-tune from this layer onwards
fine_tune_at = 100
for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

# Recompile with low learning rate
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history_fine = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=5 # Additional epochs
)

# --- 5. SAVE ---
if not os.path.exists("models"): os.makedirs("models")
model.save(MODEL_PATH)
print(f"[SUCCESS] Model Saved to {MODEL_PATH}")

# Save class names for C++
with open("classes.txt", "w") as f:
    for name in class_names:
        f.write(name + "\n")
print("[INFO] Class names saved to classes.txt")

```

To meet the requirements for TFLite Micro implementation, the MobileNetV2 model underwent full integer quantization. This process converts the model's weights and activations from 32-bit floating-point to 8-bit integers.

Table 6 Model Converter Code to TFLite Micro

```
import tensorflow as tf
import numpy as np
import os
import cv2

# --- CONFIG ---
MODEL_PATH = "models/plant_diseases_mobilenet.h5"
TFLITE_PATH = "models/plant_diseases_mobilenet_quant.tflite"
HEADER_PATH = "../model_data.h"
# We use the train folder for calibration since we don't have a dedicated test folder
DATASET_DIR = "dataset/train"
IMG_SIZE = 96

if not os.path.exists(MODEL_PATH):
    print("[ERROR] Model not found. Run training.py first.")
    exit()

# --- REPRESENTATIVE DATASET GENERATOR ---
def representative_dataset_gen():
    # Iterate through subfolders
    count = 0
    for root, dirs, files in os.walk(DATASET_DIR):
        for file in files:
            if file.lower().endswith(('.png', '.jpg', '.jpeg')):
                img_path = os.path.join(root, file)
                img = cv2.imread(img_path)
                if img is None: continue

                img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
                img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
                img = img.astype(np.float32)
                # Note: The model has Rescaling layer inside, so we feed 0-255 float
                img = np.expand_dims(img, axis=0)
                yield [img]

            count += 1
            if count >= 100: return

print("[INFO] Loading Model...")
# compile=False avoids issues if custom optimizers were used
model = tf.keras.models.load_model(MODEL_PATH, compile=False)

print("[INFO] Converting to TFLite (INT8)...")
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_dataset_gen
# Ensure full integer quantization for MCU compatibility
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
```

```

tflite_model = converter.convert()

with open(TFLITE_PATH, "wb") as f:
    f.write(tflite_model)
print(f"[SUCCESS] TFLite saved: {TFLITE_PATH}")

# --- HEX DUMP ---
def create_header(data, var_name):
    header = f"/* Generated by convert.py */\n"
    header += f"#ifndef MODEL_DATA_H\n#define MODEL_DATA_H\n"
    header += f"#include <stdint.h>\n"
    header += f"alignas(16) const unsigned char {var_name}[] = {{\n"
    for i, byte in enumerate(data):
        header += f"0x{byte:02x}, "
        if (i + 1) % 12 == 0: header += "\n"
    header += "\n};\n"
    header += f"const int {var_name}_len = {len(data)};\n"
    header += f"#endif\n"
    return header

with open(HEADER_PATH, "w") as f:
    f.write(create_header(tflite_model, "mobilenet_model"))
print(f"[SUCCESS] C Header saved: {HEADER_PATH}")

```

At the last part of implementation utilizes the TensorFlow Lite for Microcontrollers library to execute the optimized MobileNetV2 model on the STM32F746G-Discovery board.

- **Memory Allocation:** A 260 KB Tensor Arena was defined to hold the intermediate activation tensors. This size was carefully selected to accommodate the depthwise separable convolutions of MobileNetV2 without exceeding the board's SRAM.
- **Operator Registration:** Using the MicroMutableOpResolver, only the specific kernels required by the model were registered. This minimizes the memory footprint by excluding unused TFLite operators.

The output layer of the model provides a probability distribution across 16 classes.

- **Softmax Interpretation:** The code iterates through the "output -> data.int8" array to identify the class with the highest probability score.
- **Result Reporting:** The board transmits the final classification in the format "CLASS:index,score" via the Virtual COM port. This allows for immediate verification of the model's diagnostic accuracy in the field.

Table 7 Mbed Studio Code of Plant Disease Classification

```

/*
 * Final Project - Question 2
 * Model: MobileNetV2 (0.35 alpha, 96x96 input)
 * Task: Tomato Disease/Pest Classification (16 Classes)
 * Platform: STM32F746G-Discovery via Mbed Studio
 */

```

```

#include "mbed.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/schema/schema_generated.h"
#include "model_data.h"

// --- HARDWARE ---
static UnbufferedSerial pc(USBTX, USBRX, 115200);
DigitalOut led(LED1);

// --- MEMORY ---
const int kTensorArenaSize = 260 * 1024;
alignas(16) uint8_t tensor_arena[kTensorArenaSize];

// --- MODEL SETTINGS ---
const int IMAGE_WIDTH = 96;
const int IMAGE_HEIGHT = 96;
const int IMAGE_CHANNELS = 3;

// Updated for the new dataset (16 Classes)
const int NUM_CLASSES = 16;

void serial_write_str(const char* str) {
    pc.write(str, strlen(str));
}

int main() {
    led = 1; thread_sleep_for(200); led = 0; thread_sleep_for(200);

    // 1. Load Model
    const tflite::Model* model = tflite::GetModel(mobilenet_model);

    if (model->version() != TFLITE_SCHEMA_VERSION) {
        serial_write_str("[ERROR] Model Schema Version Mismatch!\n");
        while(1) { led = !led; thread_sleep_for(50); }
    }

    // 2. Register Ops
    static tflite::MicroMutableOpResolver<20> resolver;
    resolver.AddConv2D();
    resolver.AddDepthwiseConv2D();
    resolver.AddAdd();
    resolver.AddRelu();
    resolver.AddRelu6();
    resolver.AddMean();
    resolver.AddAveragePool2D();
    resolver.AddFullyConnected();
    resolver.AddSoftmax();
    resolver.AddQuantize();
    resolver.AddDequantize();
    resolver.AddReshape();
    resolver.AddPad();
    resolver.AddConcatenation();
    resolver.AddMul();
    resolver.AddSub();
    resolver.AddLogistic();

```



```

// 3. Instantiate Interpreter
static tfLite::MicroInterpreter interpreter(
    model, resolver, tensor_arena, kTensorArenaSize);

// 4. Allocate Memory
if (interpreter.AllocateTensors() != kTfLiteOk) {
    serial_write_str("[ERROR] AllocateTensors Failed!\n");
    while(1) { led = !led; thread_sleep_for(100); }
}

TfLiteTensor* input = interpreter.input(0);
TfLiteTensor* output = interpreter.output(0);

serial_write_str("--- Tomato Disease Classifier Ready ---\n");

while (true) {
    // Handshake
    char handshake = '#';
    pc.write(&handshake, 1);

    // Receive Image
    int total_bytes = input->bytes;
    int bytes_received = 0;
    int8_t* input_ptr = input->data.int8;

    while (bytes_received < total_bytes) {
        if (pc.readable()) {
            pc.read(&input_ptr[bytes_received], 1);
            bytes_received++;
        }
    }

    led = 1;
    interpreter.Invoke();
    led = 0;

    // Process Output
    int8_t* results = output->data.int8;
    int max_index = 0;
    int8_t max_score = -128;

    for (int i = 0; i < NUM_CLASSES; i++) {
        if (results[i] > max_score) {
            max_score = results[i];
            max_index = i;
        }
    }

    // Send Result: "CLASS:index, score"
    char buffer[64];
    int len = sprintf(buffer, "CLASS:%d,%d\n", max_index, max_score);
    pc.write(buffer, len);
}
}

```

Following the training and conversion phase, a comprehensive test was performed using a dedicated Python evaluation script and independent test data to verify the generalization of the models after the hardware deployment.



Figure 1 Example Image of Fugiperda Disease



Figure 2 Example Image of Healty Leaf

As the result of testing, MCU classification confusion matrix demonstrates strong diagnostic reliability after INT8 quantization. The model shows clear diagonal dominance, identifying classes such as "Caterpillars" (13 correct) and "Healthy Leaf" (13 correct) with high precision. While minor confusion exists between early-stage "Virosis" and healthy leaves, the overall performance confirms the model's ability to distinguish between 16 distinct tomato pests and diseases on-device.

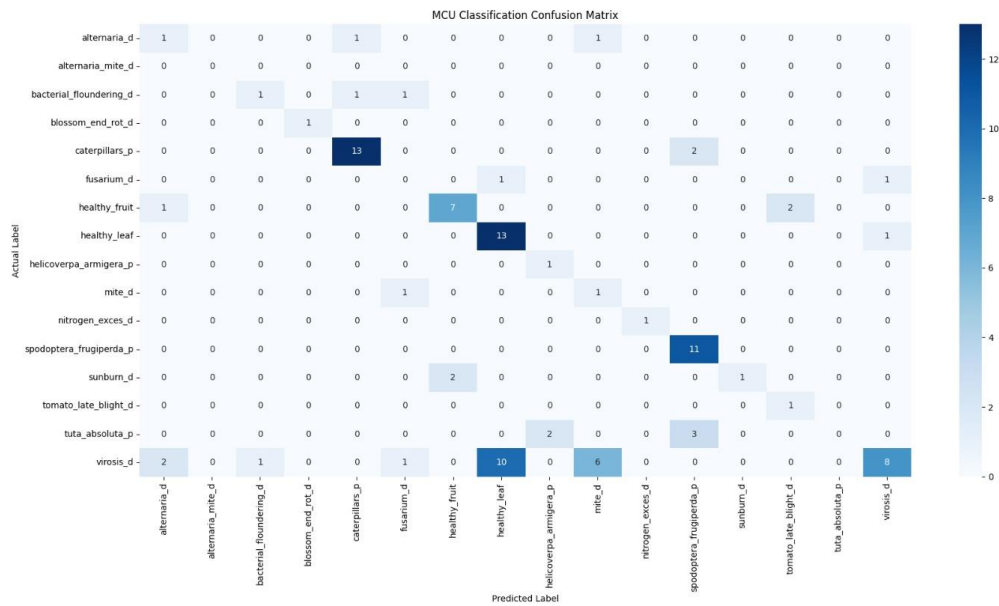


Figure 3 Confusion Matrix

Q3 – Regression Model for Air Quality Prediction on PC

For this task, the UCI Air Quality Dataset was selected. The objective is to a complex regression problem where the goal is to predict the "Ground Truth" concentration of Benzene (C_6H_6) based on the non-linear responses of several low-cost metal oxide chemical sensors. This problem is significantly more challenging than standard linear regression because chemical sensors exhibit cross-sensitivity to temperature, humidity, and other gases. The data for the air quality prediction task was obtained from the "Air Quality Data Set" hosted on the UCI Machine Learning Repository (also accessed via Kaggle).

Before training the neural network, the raw data underwent extensive cleaning to ensure model stability:

- **Handling Missing Values:** The dataset uses a placeholder value of -200 for missing or erroneous sensor readings. These were identified, replaced with NaN, and subsequently removed from the dataset to prevent the model from learning incorrect patterns.
- **Feature Selection:** Eight features were selected as inputs: five sensor responses, temperature, relative humidity and absolute humidity.
- **Feature Scaling:** Because FCNNs use gradient-based optimization, features with different scales can lead to slow convergence. Therefore standardization (Z-score normalization) was applied to all features to ensure a mean of 0 and a standard deviation of 1.

A Fully Connected Neural Network (FCNN) was implemented on the PC. The architecture was designed with a "tapered" structure (a neural network architecture where the number of neurons in each successive hidden layer decreases as you get closer to the output) to balance complexity and generalization:

- Input Layer: 8 neurons (corresponding to the 8 features).
- Hidden Layers: Three dense layers consisting of 64, 32, and 16 neurons respectively.
- Activation Function: The ReLU activation function was used in all hidden layers to introduce non-linearity, allowing the model to capture the complex chemical relationships between sensors.
- Output Layer: A single neuron with a linear activation function to output the continuous predicted value of Benzene concentration.

Optimization and training parts:

- Optimizer: The Adam (Adaptive Moment Estimation) optimizer was chosen for its efficiency in handling non-linear regression tasks and its ability to adjust learning rates for each parameter.
- Loss Function: Mean Squared Error (MSE) was used as the primary loss function to penalize larger errors more heavily during the training phase.
- Early Stopping: To prevent overfitting and ensure the model generalizes to unseen data, an Early Stopping callback was implemented. The training process monitored the “validation_loss” with a patience of 15 epochs. If the validation error failed to improve for 15 consecutive iterations, the training stopped, and the model weights were restored to the "best" historical version found during training.
- Total Epochs: A maximum of 500 epochs was defined, though the early stopping criteria typically concluded training much earlier to maintain peak performance.

Table 8 Full Code of the Regression Model

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.callbacks import EarlyStopping

# 1. LOAD AND CLEAN DATA
df = pd.read_csv('AirQuality.csv', sep=';', decimal=',')
df = df.iloc[:, 0:15].dropna(how='all')
df.replace(-200, np.nan, inplace=True)
df.dropna(inplace=True)

# 2. FEATURE SELECTION
features = ['PT08.S1(CO)', 'PT08.S2(NMHC)', 'PT08.S3(NOx)',
            'PT08.S4(NO2)', 'PT08.S5(O3)', 'T', 'RH', 'AH']
target = 'C6H6(GT)'
X = df[features]
y = df[target]
```

```

# 3. PREPROCESSING
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. DEFINE MODEL & PRINT SUMMARY
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dense(32, activation='relu'),
    Dense(16, activation='relu'),
    Dense(1)
])
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()

# 5. CONFIGURE EARLY STOPPING
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=15,
    restore_best_weights=True,
    verbose=1
)

# 6. TRAINING
print("\nTraining for up to 500 epochs...")
history = model.fit(
    X_train_scaled, y_train,
    epochs=500,
    validation_split=0.2,
    batch_size=32,
    callbacks=[early_stop],
    verbose=1
)

# 7. EVALUATION
y_pred = model.predict(X_test_scaled)
print(f"\nFinal R-squared Score: {r2_score(y_test, y_pred):.4f}")

# 8. VISUALIZATIONS
plt.figure(figsize=(10, 5))

# Graph 1: Actual vs. Predicted
plt.subplot(1, 2, 1)
plt.scatter(y_test, y_pred, alpha=0.3, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.title('Actual vs. Predicted Benzene')
plt.xlabel('Ground Truth')
plt.ylabel('Model Prediction')

# Graph 2: Loss Curve
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Training History')
plt.xlabel('Epochs')
plt.ylabel('MSE')
plt.legend()

```

```
plt.tight_layout()
plt.show()

# 9. SAVE THE MODEL
# This saves the entire model (architecture, weights, and optimizer state)
model.save('AirQuality_FCNN_Model.keras')
print("\nModel saved successfully as 'AirQuality_FCNN_Model.keras'")
```

The results demonstrate an exceptionally high-performing regression model: the Actual vs. Predicted plot shows near-perfect alignment along the 45° identity line, confirming the model's precision in capturing complex sensor non-linearities, while the loss plots confirms a stable convergence with no signs of overfitting, as the validation loss closely tracks the training loss before being halted by the early stopping mechanism at the optimal point.

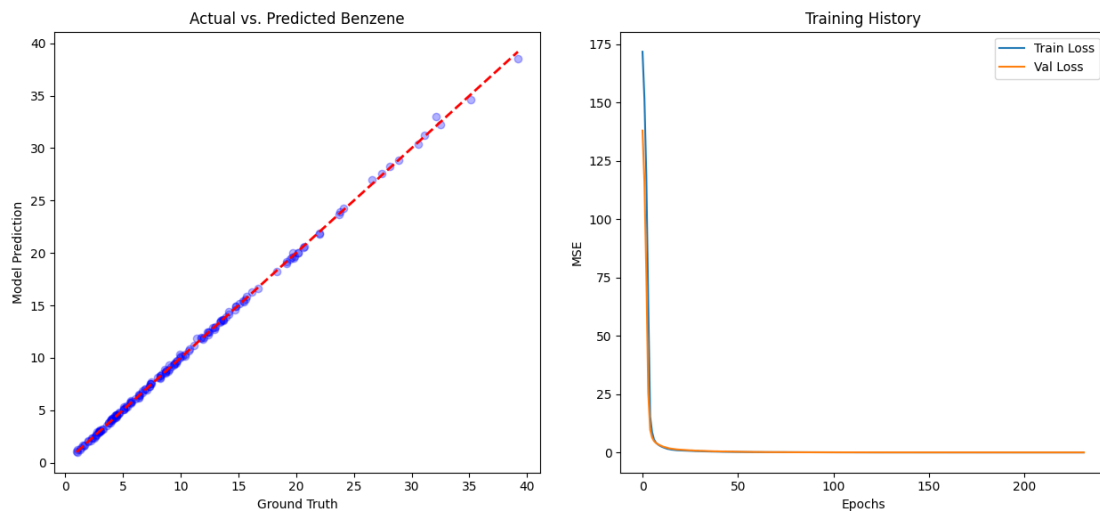


Figure 4 Results of Regression Model